The Tuition Cost and the Delivery J-Curve

When organizations adopt generative coding at scale, they frequently experience a temporary, yet measurable, dip in overall delivery stability and throughput. This initial productivity and stability dip is not a failure of the technology, but rather a budgeted "tuition cost" that organizations must pay while adapting their legacy verification systems to handle agentic output.

Responsibility Pyramid

Tiers of the Pyramid | **What matters Must** | **Agent Catalog** | **Flow** | **Core Operational Mandates**

What matters Must: Quantifies net economic value(s) after deducting human validation overhead.

Agent Catalog: Governs the portfolio of deployed copilots, workflows, and autonomous agents

Core Operational Mandates: Enforces non-negotiable guardrails

Tier 1

Control functions. These are the mechanisms that keep cost and risk bounded as autonomy grows: the Governance pillar, promotion and demotion gates policies, risk ceilings, SLOs and error budgets, the Autonomy Budget Governor, and audit trails. They are what let an agent move from HITL to HOTL safely.
Total Cost of Ownership. This is the full cost of running an agent, not just tokens or licenses. It's the same "Total Cost" used in AI Yield: operational cost, Verification Tax, Rework Tax, technical debt, and risk cost. This layer is what keeps the ROI layer honest.

Business Outcomes & ROI (metric tree below)

Value Stream, Evaluation, Governance: what matters to the business

WHAT WE MEASURE (ROI tree)
VALUE:
- lead time for change
- throughput (features per quarter)
- defect escape rate · incident MTTR
- adoption (% of teams using agents)

COST:
- token spend per feature
- platform and infrastructure
- human review hours

RISK:
- policy violations
- audit findings
- data-residency breaches

ROI = value delivered ÷ (cost + risk-adjusted loss)

Attribution: every agent action is cost-tagged and traced, so value and cost roll up per team, value stream and region.

The Strategic ROI & Yield Forecaster
Acts as an economic modeling assistant to help leadership prioritize long-term Return on Investment (ROI) over localized task speed. Operating entirely within the closed-loop delivery system, it maintains direct access to all upstream operational metrics to analyze actual portfolio returns in near real-time. Its primary focus is dynamically evaluating these comprehensive data points to determine if strategic recalculation or replanning is necessary based on actual yield. While it tracks foundational metrics like the temporary "tuition cost" of adapting legacy systems, its core mandate is providing actionable insights to continuously align enterprise investments with deployed business value.

The Local AI Pattern & Consolidation Auditor
Operates as a continuous monitor to manage the compounding complexity of localized AI tool creation. Rather than treating local AI usage as a rogue risk, it recognizes that approved AI empowers individuals to build customized tools that accelerate their specific workflows. Because these individually built tools utilize varying implementations and patterns, they introduce significant integration complexity as their outputs are pushed upstream. This persona's primary focus is tracking these fragmented local implementations, understanding their differences, and actively consolidating them into unified, common enterprise standards as they move up the vertical.

The TCO & Capacity Allocation
Acts as a dynamic executive filter to help senior leadership manage the Total Cost of Ownership (TCO) and ensure investments optimize for net capacity rather than gross generation speed. While it monitors overarching macroeconomic metrics like the delivery J-curve, its primary function is retrieving and analyzing the vast array of customizable service level agreements (SLAs), objectives (OKRs), and indicators generated across all lower delivery tiers. It continuously evaluates this systemic telemetry to filter out noisy or irrelevant data, surfacing only the specific, high-signal indicators that align directly with leadership's immediate contextual needs and strategic priorities.

The Amplified Illusion Detector
Assists in defining the rigorous "qualification vector" required to establish a Production-Qualified Change (PQC). It continuously synthesizes DORA and

Long-term Return investments: Define the overarching enterprise mandate for AI integration by prioritizing the long-term Return on Investment (ROI) over localized task speed, and establish the strategic tolerance for the budgeted "tuition cost" required to adapt legacy verification systems.

Strict Oversight Controls Over Agents: Enforce strict data-layer governance, endpoint data lineage, and prompt monitoring to mitigate the severe financial (token maxing), regulatory, and security liabilities introduced by autonomous agents operating outside of IT oversight.

Funding The J-curve: Manage the overarching Total Cost of Ownership (TCO) and secure the financial resources needed to navigate the delivery J-curve. The temporary dip in productivity and stability, ensuring the organization successfully optimizes for net capacity (long-term) rather than gross generation speed (short-term).

The Right Adoption Metrics: prevent rewarding raw adoption metrics, such as active users or token consumption, and establish the foundational mandate to optimize for "Net Delivery Capacity" (defined in The What → Net Productivity Gain). Unchecked AI generation without corresponding review capacity merely accumulates work in progress and drives up costs without delivering value.

Tier 2

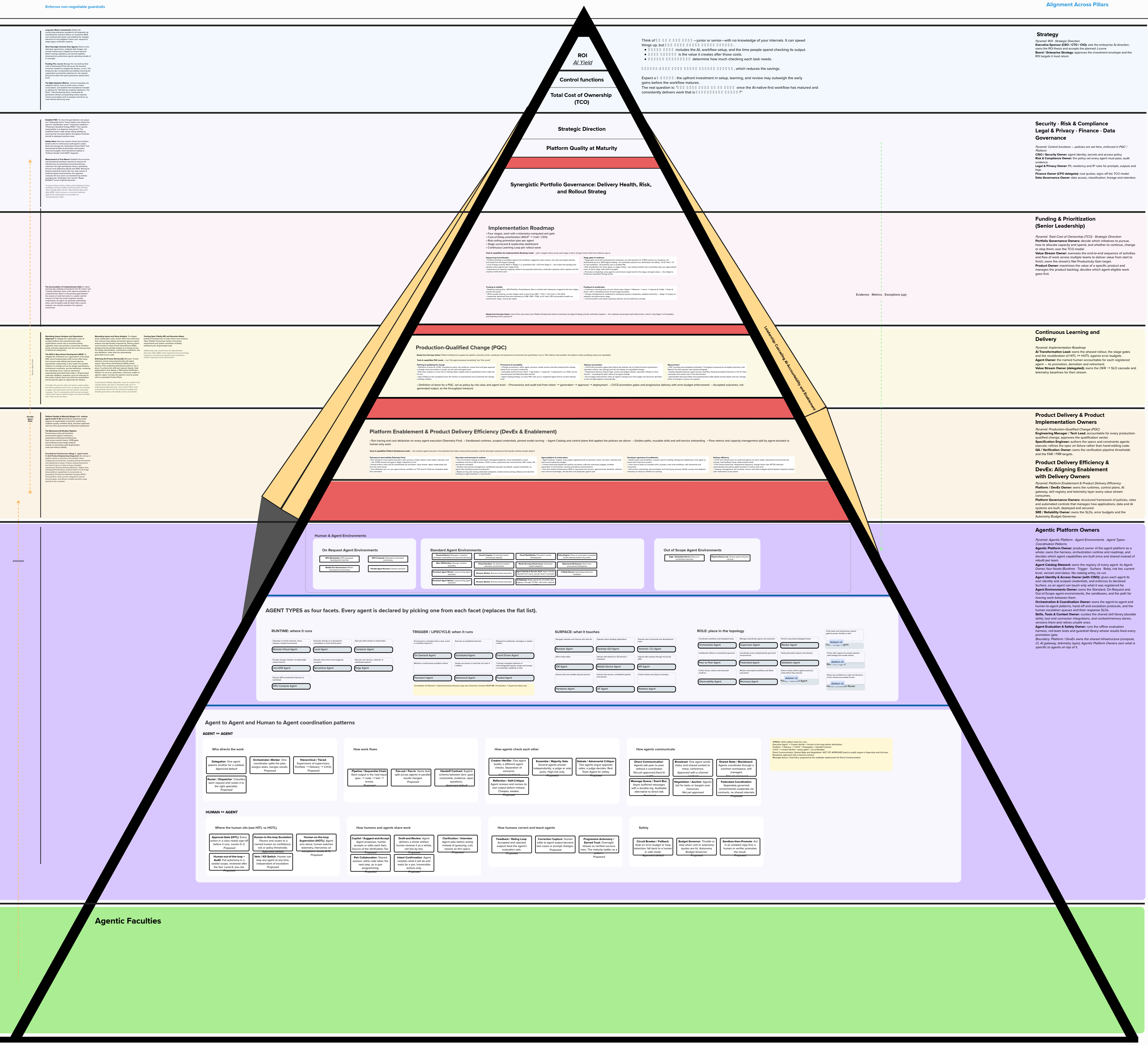
Tier 3

Tier 4

Tier 5

Agentic Platform

Tier 2



ROI
AI Yield
Control functions
Total Cost of Ownership (TCO)
Strategic Direction
Platform Quality at Maturity
Synergistic Portfolio Governance: Delivery Health, Risk, and Rollout Strateg
Leadership: Cascading AI Accountability and Enablement
Evidence · Metrics · Exceptions (up)
Agentic Faculties

Think of [illegible] —junior or senior—with no knowledge of your internals. It can speed things up, but [illegible].
- [illegible] includes the AI, workflow setup, and the time people spend checking its output.
- [illegible] is the value it creates after those costs.
- [illegible] determine how much checking each task needs.

[illegible], which reduces the savings.

Expect a [illegible]: the upfront investment in setup, learning, and review may outweigh the early gains before the workflow matures.
The real question is: "[illegible] once the AI-native-first workflow has matured and consistently delivers work that is [illegible]?"

Implementation Roadmap

- Four stages, each with a telemetry-computed exit gate
- Cost-of-Delay prioritization (WSJF → CoD / CD3)
- Risk-ceiling promotion plan per agent
- Stage scorecard & leadership dashboard
- Continuous Learning Loop per rollout wave

Tools & capabilities the Implementation Roadmap needs — plan a staged rollout, prove each stage is done, change course when the evidence says to

Sequencing & prioritization
- Portfolio backlog of candidate agents and workflows, tagged by value stream, risk class and target maturity level (built from the Agent Catalog)
- Cost of Delay scoring: WSJF in Stages 1–2, quantified CoD / CD3 from Stage 3 — this orders the backlog and decides which agents each stage funds
- Dependency & capacity mapping: platform prerequisites (telemetry, verification pipeline), team capacity and the capacity-reallocation pool

Stage gates & readiness
- Stage-gate scorecard computed from telemetry, not self-reported: S1 ≥95% traced runs, baselines, SLI dashboards by lens, 100% Agent Catalog · S2 verification pipeline live, Verification Tax falling · S3 AI Yield > 1.0 on top workflows · S4 quarterly cost vs audited P&L
- Risk classification for every agent, so stage ceiling + risk ceiling combine into a promotion plan per agent (which level, at which stage, with what oversight)
- Promotion scheduling: every agent's current level, target level for the stage, and gate status — the bridge to Production-Qualified Change below

Tracking & visibility
- Roadmap tooling (Jira / ADO Portfolio, Productboard, Aha! or similar) with milestones mapped to the four stages and the five pillars
- OKR cascade tracking, so each stage's work is tied to the OKR → SLO → SLI chain in The What
- Leadership dashboard that joins telemetry to CRM / ERP / ITSM, so AI Yield, CRV and portfolio health are reported per stage, not just per agent

Feedback & recalibration
- Continuous Learning Loop run per rollout wave: Adopt → Measure → Learn → Improve & Codify → Scale & Share, with a scheduled review at each stage boundary
- Change management & enablement: training by persona, playbooks, adoption telemetry — Stage 1 is largely an adoption and governance stage
- Communications and status-reporting cadence up the leadership cascade

Needs from the layer below: most of this only works once Platform Enablement delivers telemetry, the Agent Catalog and the verification pipeline — the roadmap cannot gate itself without them, which is why Stage 1 is Foundation and Telemetry First is priority #1.

Production-Qualified Change (PQC)

Needs from the layer below: Platform Enablement supplies the pipeline, telemetry hooks, sandboxes and ephemeral environments the qualification runs in. PQC defines what qualifies; the platform makes qualifying cheap and repeatable.

Tools & capabilities PQC needs — turn "the agent produced something" into "this counts"

Defining & qualifying the change
- Definition of done for a PQC, encoded as policy: the evidence, review level and gate approval a change must carry before it counts, by risk class and agent level
- Risk-class routing, so a low-risk L2 change takes a lighter path to qualification than a high-risk L5 one
- Spec Fidelity as the acceptance test: the fraction of requirements and constraints the change verifiably satisfies

Provenance & traceability
- Change provenance: which agent, persona, model version and task produced the change, linked to its run trace in telemetry
- Immutable audit trail from intent → generation → approval → deployment, so a PQC can be reconstructed and defended after the fact
- Agent Catalog linkage, so every PQC rolls up to a registered agent and its current maturity level

Delivery & promotion
- CI/CD with promotion gates that enforce the policies set in Control functions (promotion / demotion criteria, risk ceilings) and will not release an unqualified change
- Progressive delivery: feature flags, canary and staged rollouts, automatic rollback on SLO breach — bounding the blast radius of an agent change
- Error-budget enforcement: when an agent's changes burn the budget, the Governor demotes it and oversight tightens automatically

Measurement
- PQC counting and acceptance telemetry: Throughput measured as accepted outcomes, with Rework Tax and rejection rate tracked alongside
- Quality-adjusted yield per agent and per workflow, feeding Accepted Outcomes in the AI Yield numerator and rework cost in the denominator
- Specification Execution Rate and comprehension-debt signals (review depth, time per change, share of changes a human can explain)

• Definition of done for a PQC, set as policy by risk class and agent level · • Provenance and audit trail from intent → generation → approval → deployment · • CI/CD promotion gates and progressive delivery with error-budget enforcement · • Accepted outcomes, not generated output, as the throughput measure

Platform Enablement & Product Delivery Efficiency (DevEx & Enablement)

• Run tracing and cost attribution on every agent execution (Telemetry First) · • Sandboxed runtimes, scoped credentials, pinned model serving · • Agent Catalog and control plane that applies the policies set above · • Golden paths, reusable skills and self-service onboarding · • Flow metrics and capacity management split by agent-assisted vs human-only work

Tools & capabilities Platform Enablement needs — the runtimes agents execute in, the telemetry that makes measurement possible, and the developer experience that decides whether people adopt it

Telemetry & observability (Telemetry First)
- Run tracing for every agent execution: task, persona, model, tokens, tools called, outcome, cost — the ≥95% traced-runs gate in Stage 1 depends on this
- Shared metrics store and SLI dashboards by lens (team, value stream, agent, leadership), fed from the same traces
- Cost attribution per run, per agent and per workflow, so TCO and AI Yield are computed rather than estimated

Execution environments & runtimes
- The environment catalog on the board: managed endpoints, cloud workstations, virtual sandboxes and micro-VM runtimes, CI/CD runners, ephemeral dev environments, GPU nodes, HIL and device emulators
- Isolation and secrets management: sandboxed execution by default, scoped credentials, no agent with standing access to production
- Model serving and routing: dedicated endpoints, model-version pinning, fallback and rate-limit handling so agent behavior is reproducible

Agent platform & control plane
- Agent Catalog / registry: every agent registered with its persona, owner, risk class, maturity level and current autonomy budget
- Control plane that applies the policies set above: enforces autonomy budgets, throttles generation on SLO breach, records promotions and demotions
- Tool and context infrastructure: MCP or equivalent tool servers, approved tool allowlists, retrieval over internal knowledge, shared skills and playbooks agents load

Developer experience & enablement
- Golden paths and templates: a paved road for building, testing and registering a new agent so teams don't start from scratch
- Inner-source library of reusable skills, prompts, evals and workflows, with ownership and versioning
- Self-service onboarding, documentation and training by persona; DevEx surveys and adoption telemetry to measure whether it's working

Delivery efficiency
- CI/CD and infrastructure-as-code that agents can drive safely: ephemeral environments per change, fast feedback loops, deterministic builds
- Flow metrics (lead time, deployment frequency, change failure rate, MTTR) collected automatically and split by agent-assisted vs human-only work
- Capacity management: the compute, licence and token budgets behind System Capacity Control, with reallocation across teams

Human & Agent Environments

On Request Agent Environments
- **GPU Workstation:** GPU-equipped development machine
- **GPU Compute:** Dedicated accelerated processing
- **Mobile Dev Environment:** Mobile development and testing
- **Parallel Agent Runners:** Parallel execution

Standard Agent Environments
- **Personal Device:** Managed, compliant developer workstations (no personal devices)
- **Cloud Compute:** On-demand elastic processing capacity
- **Cloud WorkStation:** Persistent remote development
- **Policy Engine:** Policy-as-code gates evaluated by the Harness before any action
- **Mico VM Runtime:** Stronger isolation boundary
- **Virtual Sandbox:** On-demand isolated execution environments
- **Model Serving Infrastructure:** Dedicated model endpoints
- **Ephemeral Workspace:** Short-lived provisioned workspaces
- **Persistent Agent Worker:** Long-running agent execution
- **Browser Worker:** Browser-based execution
- **Agent Identity & Secrets Vault:** Agent identity separate from users; secrets never in prompts
- **CI Build Runner:** Automated build and validation
- **Persistent Agent Worker:** Long-running agent execution
- **Browser Worker:** Browser-based execution
- **AI Gateway:** Single egress for all model calls: logging, cost tags, PII filter, rate limits, regional

Out of Scope Agent Environments
- **Edge / Embedded Device:** Resource-constrained execution
- **Physical Device Lab:** Testing against physical devices

AGENT TYPES as four facets. Every agent is declared by picking one from each facet (replaces the flat list).

RUNTIME: where it runs
- Remote Virtual Agent — Operates in remote machine, cloud instance, hosted environment
- Local Agent — Executes directly on a developer's workstation or local infrastructure
- Container Agent — Execute within Docker or Kubernetes
- microVM Agent — Provide stronger isolation via lightweight virtual machine
- Serverless Agent — Execute short-lived, event-triggered functions
- Edge Agent — Execute near devices, networks or distributed systems
- GPU Compute Agent — Execute GPU-accelerated inference or workloads

TRIGGER / LIFECYCLE: when it runs
- On-Demand Agent — Provisioned or activated when a task, event or workflow requires it
- Scheduled Agent — Execute on predefined intervals
- Event Driven Agent — Respond to webhooks, messages or system events
- Persistent Agent — Maintain a continuously available runtime
- Ephemeral Agent — Single-use session or task that can work in isolation
- Pooled Agent — Centrally managed collection of interchangeable agents; assign work based on availability, capability or load

Consolidate: On-Demand ≈ Ephemeral (same lifecycle, keep one). Serverless moved to RUNTIME. Orchestrator ≈ Supervisor (keep one).

SURFACE: what it touches
- Browser Agent — Navigate websites and interact with web UIs
- Desktop GUI Agent — Operate native desktop applications
- Terminal / CLI Agent — Execute shell commands and development tools
- IDE Agent — Work inside editor
- Mobile Device Agent — Interact with Android or iOS devices / emulators
- API Agent — Interact with systems through structured APIs
- Hardware Agent — Interact with and validate physical devices
- IoT Agent — Interact with sensors, embedded systems and devices
- Robotics Agent — Control robots and physical actuators

ROLE: place in the topology
- Orchestrator Agent — Coordinate workflows and delegated tasks
- Supervisor Agent — Manage subordinate agents and execution
- Worker Agent — Perform specialized delegated tasks
- Cost Budget Agent — Track token and infrastructure spend against quotas; throttle or alert. Added v2
- Peer to Peer Agent — Collaborate without a centralized supervisor
- Federated Agent — Coordinate across independently governed environments
- Validation Agent — Verify generated outputs and behavior
- Red-Team Agent — Probe other agents for prompt injection, data leakage and unsafe actions. Added v2
- Observability Agent — Collect traces, metrics and execution evidence
- Recovery Agent — Restore interrupted workflows and failed executions
- Policy Guardrail Agent — Allow or deny actions against policy-as-code before they execute. Added v2
- Human-Escalation Router — Route low-confidence or high-risk decisions to the named accountable human. Added v2

Agent to Agent and Human to Agent coordination patterns

AGENT ↔ AGENT

Who directs the work
- **Delegation**: One agent spawns another for a subtask. *Approved default*
- **Orchestrator–Worker**: One coordinator splits the plan, assigns tasks, merges results. *Proposed*
- **Hierarchical / Tiered**: Supervisors of supervisors: Portfolio → Delivery → CI/CD. *Proposed*
- **Router / Dispatcher**: Classifies each request and routes it to the right specialist. *Proposed*

How work flows
- **Pipeline / Sequential Chain**: Each output is the next input: spec → code → test → review. *Proposed*
- **Fan-out / Fan-in**: Same task split across agents in parallel; results merged. *Proposed*
- **Handoff Contract**: Explicit schema between tiers: goal, constraints, evidence, open questions. *Approved default*

How agents check each other
- **Creator–Verifier**: One agent builds, a different agent checks. Separation of concerns. *Approved default*
- **Ensemble / Majority Vote**: Several agents answer independently; a judge or vote picks. High-risk only. *Proposed*
- **Debate / Adversarial Critique**: Two agents argue opposite sides; a judge decides. Red-Team Agent for safety. *Proposed*
- **Reflection / Self-Critique**: Agent reviews and revises its own output before release. Cheaper, weaker. *Proposed*

How agents communicate
- **Direct Communication**: Agents talk peer to peer without a coordinator. *Not yet approved (hard to audit)*
- **Broadcast**: One agent sends status and shared context to many; coherence. *Approved with a channel contract*
- **Shared State / Blackboard**: Agents coordinate through a common workspace, self-managed. *Not yet approved*
- **Message Queue / Event Bus**: Async buffered messages with a durable log. Auditable alternative to direct talk. *Proposed*
- **Negotiation / Auction**: Agents bid for tasks or bargain over resources. *Not yet approved*
- **Federated Coordination**: Separately governed environments cooperate via contracts, no shared internals. *Proposed*

WIRING: which pattern each tier uses
Executive Agent → Creator-Verifier + Human-in-the-loop before distribution.
Portfolio → Delivery → CI/CD → Delegation + Handoff Contract.
CI/CD → Creator-Verifier + policy gates + Circuit Breaker.
Direct Communication, Shared State and Negotiation: NOT YET APPROVED (hard to audit); require a Supervisor and full trace.
Broadcast: approved with a channel contract.
Message Queue / Event Bus: proposed as the auditable replacement for Direct Communication.

HUMAN ↔ AGENT

Where the human sits (see HITL vs HOTL)
- **Approval Gate (HITL)**: Every action in a class needs sign-off before it runs. Levels 0–3. *Proposed*
- **Human-in-the-loop Escalation**: Pauses and routes to a named human on confidence, risk or policy thresholds. *Approved default*
- **Human-on-the-loop Supervision (HOTL)**: Agent acts alone; human watches telemetry, intervenes on exceptions. Levels 4–5. *Proposed*
- **Human-out-of-the-loop + Audit**: Full autonomy in a sealed scope, reviewed after the fact. Level 6, low risk. *Proposed*
- **Veto / Kill Switch**: Human can stop any agent at any time, independent of escalation. *Proposed*

How humans and agents share work
- **Copilot / Suggest-and-Accept**: Agent proposes, human accepts or edits each item. Source of the Verification Tax. *Proposed*
- **Draft-and-Review**: Agent delivers a whole artifact; human reviews it as a whole, not line by line. *Proposed*
- **Clarification / Interview**: Agent asks before acting instead of guessing; cuts rework on thin specs. *Proposed*
- **Pair Collaboration**: Shared session; either side takes the next step, as in pair programming. *Proposed*
- **Intent Confirmation**: Agent restates what it will do and waits for a yes. Irreversible actions only. *Proposed*

How humans correct and teach agents
- **Feedback / Rating Loop**: Accepted and rejected outputs feed the agent's evaluation sets. *Proposed*
- **Correction Capture**: Human edits to agent output become test cases or prompt changes. *Proposed*
- **Progressive Autonomy / Earned Trust**: Oversight relaxes as verified success rises. The maturity ladder as a pattern. *Proposed*

Safety
- **Circuit Breaker / Fallback**: Stop on error budget or loop detection; fall back to a human or safe mode. *Approved default*
- **Budget Governor**: Throttle or stop when cost or autonomy quotas are hit. Autonomy Budget Governor. *Proposed*
- **Sandbox-then-Promote**: Act in an isolated copy first; a human or verifier promotes the result. *Proposed*

Alignment Across Pillars

Strategy
Pyramid: ROI · Strategic Direction
Executive Sponsor (CEO / CTO / CIO): sets the enterprise AI direction, owns the ROI thesis and accepts the planned J-curve.
Board / Enterprise Strategy: approves the investment envelope and the ROI targets it must return.

Security · Risk & Compliance Legal & Privacy · Finance · Data Governance
Pyramid: Control functions — policies are set here, enforced in PQC / Platform
CISO / Security Owner: agent identity, secrets and access policy.
Risk & Compliance Owner: the policy set every agent must pass; audit evidence.
Legal & Privacy Owner: PII, residency and IP rules for prompts, outputs and logs.
Finance Owner (CFO delegate): cost quotas; signs off the TCO model.
Data Governance Owner: data access, classification, lineage and retention.

Funding & Prioritization (Senior Leadership)
Pyramid: Total Cost of Ownership (TCO) · Strategic Direction
Portfolio Governance Owners: decide which initiatives to pursue, how to allocate capacity and spend, and whether to continue, change or stop them; own the TCO model.
Value Stream Owner: oversees the end-to-end sequence of activities and flow of work across multiple teams to deliver value from start to finish; owns the stream's Net Productivity Gain target.
Product Owner: maximizes the value of a specific product and manages the product backlog; decides which agent-eligible work goes first.

Continuous Learning and Delivery
Pyramid: Implementation Roadmap
AI Transformation Lead: owns the phased rollout, the stage gates and the recalibration of HITL ↔ HOTL against error budgets.
Agent Owner: the named human accountable for each registered agent — its promotion, demotion and retirement.
Value Stream Owner (delegated): owns the OKR → SLO cascade and telemetry baselines for their stream.

Product Delivery & Product Implementation Owners
Pyramid: Production-Qualified Change (PQC)
Engineering Manager / Tech Lead: accountable for every production-qualified change; approves the qualification vector.
Specification Engineer: authors the specs and constraints agents execute; refines the spec on failure rather than hand-editing code.
QA / Verification Owner: owns the verification pipeline thresholds and the FAR / FRR targets.

Product Delivery Efficiency & DevEx: Aligning Enablement with Delivery Owners
Pyramid: Platform Enablement & Product Delivery Efficiency
Platform / DevEx Owner: owns the runtimes, control plane, AI gateway, skill registry and telemetry layer every value stream consumes.
Platform Governance Owners: structured framework of policies, roles and automated controls that manages how applications, data and AI systems are built, deployed and secured.
SRE / Reliability Owner: owns the SLOs, error budgets and the Autonomy Budget Governor.

Agentic Platform Owners
Pyramid: Agentic Platform · Agent Environments · Agent Types · Coordination Patterns
Agentic Platform Owner: product owner of the agent platform as a whole; owns the harness, orchestration runtime and roadmap, and decides which agent capabilities are built once and shared instead of rebuilt per team.
Agent Catalog Steward: owns the registry of every agent: its Agent Owner, four facets (Runtime · Trigger · Surface · Role), risk tier, current level, version and status. No catalog entry, no run.
Agent Identity & Access Owner (with CISO): gives each agent its own identity and scoped credentials, and enforces its declared Surface, so an agent can touch only what it was registered for.
Agent Environments Owner: owns the Standard, On-Request and Out-of-Scope agent environments, the sandboxes, and the path for moving work between them.
Orchestration & Coordination Owner: owns the agent-to-agent and human-to-agent patterns, hand-off and escalation protocols, and the human escalation queues and their response SLOs.
Skills, Tools & Context Owner: curates the shared skill library (durable skills), tool and connector integrations, and context/memory stores; versions them and retires unsafe ones.
Agent Evaluation & Safety Owner: runs the offline evaluation harness, red-team tests and guardrail library whose results feed every promotion gate.
Boundary: Platform / DevEx owns the shared infrastructure (compute, CI, AI gateway, telemetry layer); Agentic Platform Owners own what is specific to agents on top of it.

Ownership and Delegation

The Ownership and Delegation domain governs human accountability, skill progression, and the dynamic allocation of decision rights between humans and machines. To eliminate governance ambiguity, the framework enforces the Single pipeline, Single Number Rule

Human Responsibility

Decisions only a named human can make (never delegated to an agent)

1 · Set intent and risk appetite
Executive Sponsor sets direction, ROI thesis and risk tolerance. Control functions turn it into policy (security, compliance, privacy, cost, data).

2 · Register an agent
No agent runs without a named Agent Owner, a declared risk tier, its four facets (Runtime · Trigger · Surface · Role) and an Agent Catalog entry.

3 · Promote an agent one level
Agent Owner proposes with evidence. AI Transformation Lead approves within the stage ceiling. Risk & Compliance must also sign for high-risk agents. Promotion is never automatic.

4 · Grant an exception above the risk ceiling
Risk & Compliance Owner + CISO, time-boxed, recorded in the audit trail. The Agent Owner cannot grant it.

5 · Change an agent's cost quota
Finance Owner, on request from the Value Stream Owner.

6 · Retire an agent or start version N+1
Agent Owner decides after Periodic Review; the Value Stream Owner is consulted.

7 · Accept a production change
Engineering Manager / Tech Lead stays accountable for every Production-Qualified Change, whoever (or whatever) wrote it.

8 · Own the spec
Specification Engineer authors specs and constraints. On failure, fix the spec, not the generated code.

9 · Pull the plug
Agent Owner, SRE on-call or CISO can take any agent offline at once. No approval needed to stop; restarting needs the Agent Owner.

10 · Appeal an automatic demotion
Agent Owner may appeal to the AI Transformation Lead, with evidence. The demotion stays in force while the appeal runs.

Skill progression (people, not agents)
Reviewer → Specification Engineer → Agent Owner → Value Stream Owner. Each step is earned through Proficiency (pillar 2) evidence and tracked in SPACE. Reviewing agent output well comes before owning agents.

Common / Shared Responsibility

Where humans and agents share a decision, and who wins when they collide

Decision rights (Decides · Consulted · Enforced by)
- Promote one level · AI Transformation Lead, on the Agent Owner's evidence · Risk & Compliance (high risk) · stage gate in PQC
- Demote · Autonomy Budget Governor (automatic) or Agent Owner · SRE · control plane
- Take offline · Agent Owner, SRE on-call or CISO · nobody needs to approve · control plane kill switch
- Exception above risk ceiling · Risk & Compliance + CISO · Agent Owner · policy engine, time-boxed
- Cost quota change · Finance Owner · Value Stream Owner · AI gateway
- Retire / version N+1 · Agent Owner · Value Stream Owner · Agent Catalog
- HITL ↔ HOTL switch · AI Transformation Lead · SRE (error budget) · control plane

Precedence: who wins
1. Policy (Control functions) beats everything below it.
2. Stage ceiling and risk ceiling beat any owner's request.
3. Automatic demotion stands; it can be appealed, never overridden.
4. Stop beats go: any authorized pause wins until the Agent Owner restarts.
5. Within those bounds, the Agent Owner decides.

Source of truth when sources disagree (proposed)
- What the agent should do → the spec wins.
- What is actually happening → production telemetry wins.
- Code is evidence, never the authority over the spec.
- Spec and production conflict → the agent stops and escalates to the Specification Engineer.

Shared accountability
- The agent produces the evidence; the human owns the outcome.
- One accountable owner per metric. Others may contribute, but only one person answers for each number.
- Every escalation lands in a named human's queue with a response SLO, so an agent never waits on "someone".
- Autonomy is earned and re-earned: levels and oversight are reviewed at each Periodic Review against error budgets and Quality Yield.

Agent Responsibility

What an agent may decide on its own: bounded by its level, scope and budget

Delegation by maturity level
L0–2 · Suggest only. A human takes every action.
L3 · Acts, but a human approves each change before it lands (HITL).
L4–5 · Acts on its own inside its scope; a human watches and samples (HOTL).
L6 · Acts autonomously, low-risk work only, with a full audit trail.
Ceiling = the lower of the stage ceiling and the risk ceiling.

Always does
- Stays inside its declared Surface (what it touches) and its autonomy and cost budget.
- Makes targeted, localized changes; no unprompted refactors or full regeneration.
- Leaves evidence: every run traced, with inputs, outputs and decisions.
- Checks before it acts (side-effect ledger) and checkpoints its state, so a resume never repeats work.
- Reports its own failures and exceptions upward.

Must escalate to a human when
- Sources of truth disagree (see the precedence rule in Shared).
- The spec is ambiguous or missing.
- An action would exceed its level, scope, budget or risk tier.
- Confidence or a verification gate falls below threshold.
- It touches PII, secrets or a new external system.

Never does
- Change its own permissions, budget, level or policy.
- Approve or promote itself, or another agent it depends on.
- Edit the spec or tests it is judged against.
- Override a pause, kill switch or automatic demotion.

Agent-to-agent delegation
An agent can only hand work to an agent at the same or a lower level. The receiving agent inherits the stricter scope. The orchestrator's Agent Owner stays accountable for the whole chain.

Core Responsibilities

Set the enterprise direction for AI, decide where and how much to invest, and own the return. They fund AI as a long-term capability, accept a planned short-term dip in delivery (the J-curve) as the cost of adoption, and set the organization's tolerance for risk, compliance and cost. Success is judged by net business value delivered, not by activity, adoption or speed.

Translate overarching delivery outcomes into rigid operational metrics, specifically Service Level Objectives (SLOs) and OKRs. They manage the "Verification Tax" and the "repair multiplier" across their engineering teams. They track "Spec Fidelity (SF)" and the "Specification Execution Rate (SER)" while actively auditing portfolios to prevent the catastrophic accumulation of "comprehension debt".

Establish the dynamic feedback loops that synthesize software delivery telemetry with industrial lean manufacturing principles like Overall Effectiveness (OE), ensuring the organizational delivery systems adapt as AI capabilities and codebase complexities evolve.

Responsible for engineering near-deterministic AI code generation by enforcing precise architectural boundaries that restrict agents to executing only targeted, localized fixes. Ensuring the AI does not regenerate entire artifacts or introduce unprompted refactoring that overwhelms testing systems. Concurrently, this role utilizes a highly observable *Verification Pipeline* to continuously surface critical delivery telemetry, establishing a closed feedback loop between investment, implementation, and reinvestment, and dynamically align operational metrics directly with the actual deployed value proposition.

Responsible for enhancing the foundational infrastructure, localized context engineering capabilities, and implementing "Verification Pipeline" required to safely scale agentic workflows. They act as the automated auditors of the delivery system by implementing strict pre-commit checks, CI gates, and continuous fuzzing to mitigate the "verification tax" without overwhelming human reviewers. Furthermore, they manage the developer experience (DevEx) by operating an automated Control Plane that dynamically throttles AI autonomy budgets whenever delivery stability or Quality Yield degrades, ensuring that platform enablement is inextricably linked to reliable product delivery.

Telemetry

Rather than allowing telemetry to remain trapped in infrastructure logs, the operating model enforces a bi-directional governance using Service Level Indicators (SLIs), Service Level Objectives (SLOs), Service Level Agreements (SLAs), and Objectives and Key Results (OKRs)

Product & Value Stream Owners

Continuous Learning and Delivery

Product Delivery & Product Implementation

Product Delivery Efficiency & DevEx: Aligning Enablement with Delivery

Framework Template

Shared Capability Map (proposed)

Where to share, repeat and reuse

Proposed restructure of the Agent Persona column onto the shared platform, following the Scaling Model: one shared platform, N value-stream instances.

SHAREABLE · one instance, consumed by every value stream
REPEATABLE · template instantiated per value stream
REUSABLE · versioned asset pulled into many agents and skills

VALUE-STREAM INSTANCES · REPEATABLE — each copies the tier template with its own owners, metrics and targets

Value stream A
Executive · Portfolio · Delivery · CI/CD agent tiers
Own owners, OKRs and metric targets
Consumes the shared capabilities below

AGENT PERSONA VIEWS · REUSABLE

Leadership & ROI
ROI & Yield Forecaster · TCO & Capacity Allocation · Grassroots ROI & Investment Informer

Delivery health
Amplified Illusion Detector · Comprehension Debt & SLO Tracker · PDE Telemetry Synthesizer

SHARED CAPABILITIES · SHAREABLE — one instance each, owned by Platform & DevEx, funded centrally

1 · Delivery Telemetry & Metrics Layer
DORA / SPACE plus SQY, PDE, Production-Qualified Yield and Net Delivery Capacity (defined in Net Productivity Gain, The What), computed once from Harness traces and cost tags.
Serves: Leadership & ROI, Delivery health views

5 · Identity, Ownership & SoD
Identity Broker + Policy Engine + one ownership / resource registry. Creator ≠ approver enforced the same way everywhere.
Serves: Governance views

FOUNDATION · SHAREABLE — SDK / Harness · Model Gateway · Identity Broker · Policy Engine · Runtime catalog · Coordination patterns
Harness guarantees stated once for every tier instead of repeated on each tier card: trace coverage · session & transaction IDs · replay · golden evaluation sets · cost tags · policy-as-code gates · kill switch

CONTROL FUNCTIONS govern every instance with one policy set: Security · Risk & Compliance · Legal & Privacy · Finance · Data Governance

REUSABLE ASSETS
Versioned, change-managed, pulled into any value stream or agent

- **Tier contract card**: purpose · inputs · outputs · decision rights · human gate · metrics · owner
- **Durable-skill template**: externalized state, idempotency ledger, HITL suspend, exit gates
- **Handoff contract schema**: goal · constraints · evidence · open questions
- **Metric definitions**: SQY · PDE · Production-Qualified Yield · Net Delivery Capacity (source of truth: Net Productivity Gain)
- **Golden evaluation sets** per value stream
- **Policy-as-code bundles**: separation of duties, data classification, residency
- **Coordination patterns**: Creator-Verifier · HITL escalation · Circuit breaker · Handoff
- **Prompt and skill versions** under change management

Promotion path
1. Grassroots tool or skill built locally
2. Registered and versioned in the Skill & Tool Registry → **REUSABLE**
3. Adopted by 2+ value streams
4. Promoted into the platform, owned by Platform & DevEx → **SHAREABLE**